

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería en Software



**IMPLEMENTACIÓN Y COMPARACIÓN DE
COMUNICACIÓN DISTRIBUIDA MEDIANTE
TCP Y gRPC UTILIZANDO RELOJES
LÓGICOS DE LAMPORT**

Asignatura: Aplicaciones Distribuidas

Equipo C – Integrantes:

Farinango Guandinango Freddy Vladimir

Gaibor Rodríguez Jeremy Ruperto

Villamarin Cuenca Ivan Andres

Docente:

Ing. Gleiston Guerrero

Período Académico:

2026

Quevedo – Ecuador

Índice

RESUMEN	2
ABSTRACT	3
1. INTRODUCCIÓN	4
2. FUNDAMENTO TEÓRICO	4
2.1. Sistemas Distribuidos	4
2.2. Comunicación mediante Sockets TCP	4
2.3. Remote Procedure Call (RPC) y gRPC	5
2.4. Protocol Buffers	5
2.5. Relojes Lógicos de Lamport	5
2.6. Medición de Latencia	6
3. DESARROLLO DEL PROYECTO	7
3.1. Arquitectura General	7
3.2. Implementación TCP	8
3.3. Implementación gRPC	8
3.4. Configuración de Nodos	8
4. RESULTADOS EXPERIMENTALES	8
4.1. Intercambios de Mensajes y Latencia Promedio	9
4.2. Análisis Estadístico	10
4.3. Análisis del Boxplot Comparativo	10
5. CONCLUSIONES	11
5.1. Freddy Vladimir Farinango Guandinango	11
5.2. Jeremy Ruperto Gaibor Rodríguez	12
5.3. Iván Andrés Villamarin Cuenca	12
6. BIBLIOGRAFIA	12
7. ANEXOS	14

RESUMEN

Los sistemas distribuidos desempeñan un papel fundamental en el desarrollo de aplicaciones modernas, permitiendo que múltiples dispositivos compartan información y recursos de forma coordinada. Dentro de este contexto, la comunicación entre procesos distribuidos constituye uno de los elementos más críticos para garantizar el correcto intercambio de información [1].

Existen diversas tecnologías para este propósito: los sockets TCP proporcionan comunicación confiable basada en conexiones, mientras que gRPC ofrece una arquitectura moderna de llamadas remotas de alto rendimiento utilizando Protocol Buffers y HTTP/2 [4], [5]. Adicionalmente, en sistemas distribuidos es necesario mantener un orden lógico de eventos sin depender del reloj físico de cada dispositivo; para ello se utilizan los relojes lógicos de Lamport, ampliamente empleados para sincronización lógica y análisis de concurrencia distribuida [1], [2].

El presente proyecto implementa y compara TCP y gRPC utilizando relojes de Lamport en un entorno de tres nodos, analizando rendimiento, latencia e identificando las ventajas de cada tecnología en sistemas distribuidos modernos.

ABSTRACT

The present project implements and compares two widely used distributed communication technologies in the context of distributed applications: TCP sockets and gRPC, also integrating the use of Lamport logical clocks to achieve event synchronization among multiple nodes within a distributed environment. For the development of this practice, three independent nodes were configured, capable of sending, receiving, and processing messages concurrently, updating their logical timestamps according to the rules established by Lamport's algorithm. This made it possible to simulate the behavior of real distributed systems and analyze how the logical order of events is maintained without relying on synchronized physical clocks.

In the TCP-based implementation, flow-oriented communication through sockets was used, employing structured messages in JSON format for information exchange between nodes. On the other hand, the gRPC implementation used Protocol Buffers as an efficient serialization mechanism and formal definition of remote services. Both technologies were subjected to different message exchange and latency measurement tests, evaluating important metrics such as mean, median, standard deviation, and the 95th percentile, with the objective of analyzing the performance, stability, and efficiency of each distributed communication approach.

1. INTRODUCCIÓN

Los sistemas distribuidos desempeñan un papel fundamental en el desarrollo de aplicaciones modernas, permitiendo que múltiples dispositivos compartan información y recursos de forma coordinada. Dentro de este contexto, la comunicación entre procesos distribuidos constituye uno de los elementos más críticos para garantizar el correcto intercambio de información.

Existen diversas tecnologías para este propósito: los sockets TCP proporcionan comunicación confiable basada en conexiones, mientras que gRPC ofrece una arquitectura moderna de llamadas remotas de alto rendimiento utilizando Protocol Buffers. Adicionalmente, en sistemas distribuidos es necesario mantener un orden lógico de eventos sin depender del reloj físico de cada dispositivo; para ello se utilizan los relojes lógicos de Lamport.

El presente proyecto implementa y compara TCP y gRPC utilizando relojes de Lamport en un entorno de tres nodos, analizando rendimiento, latencia e identificando las ventajas de cada tecnología en sistemas distribuidos modernos.

2. FUNDAMENTO TEÓRICO

2.1. Sistemas Distribuidos

Un sistema distribuido es un conjunto de computadoras independientes que trabajan coordinadamente para proporcionar un servicio o resolver un problema común. Se caracterizan por ofrecer escalabilidad, tolerancia a fallos, disponibilidad y mejor aprovechamiento de recursos. Uno de sus principales desafíos es la sincronización entre procesos y el intercambio correcto de información entre nodos [8].

Actualmente, arquitecturas modernas como microservicios, computación en la nube y sistemas IoT dependen ampliamente de modelos distribuidos para garantizar flexibilidad y escalabilidad [6]. En este contexto, tecnologías como TCP y gRPC resultan fundamentales para la comunicación eficiente entre servicios.

2.2. Comunicación mediante Sockets TCP

Los sockets TCP permiten el intercambio de datos entre procesos mediante un protocolo orientado a conexión que garantiza entrega confiable y ordenada. En una arquitectura cliente-servidor, el servidor escucha conexiones entrantes en un puerto específico y ambas partes intercambian mensajes de manera continua. Sus ventajas incluyen confiabilidad,

control de congestión y verificación de errores; sus limitaciones están relacionadas con el consumo de recursos en sistemas de gran escala.

2.3. Remote Procedure Call (RPC) y gRPC

RPC es un mecanismo que permite ejecutar procedimientos ubicados en otro dispositivo de la red como si fueran llamadas locales, simplificando el desarrollo distribuido. gRPC, desarrollado por Google, representa una de las implementaciones modernas más utilizadas, ya que emplea Protocol Buffers para serialización eficiente y HTTP/2 como protocolo de transporte [4].

Diversos estudios recientes demuestran que gRPC presenta ventajas importantes en latencia, eficiencia de serialización y estabilidad frente a tecnologías tradicionales basadas en REST y JSON [4], [5]. Además, soporta comunicación bidireccional, multiplexación y generación automática de código cliente-servidor mediante archivos .proto.

2.4. Protocol Buffers

Protocol Buffers (Protobuf) es un mecanismo de serialización desarrollado por Google que permite estructurar y transmitir datos de forma eficiente. Comparado con formatos como JSON o XML, Protobuf reduce el tamaño de los mensajes y mejora la velocidad de procesamiento [4], [5].

Esta tecnología constituye la base de serialización de gRPC y resulta especialmente útil en sistemas distribuidos y arquitecturas de microservicios donde la eficiencia de transmisión representa un factor crítico de rendimiento.

2.5. Relojes Lógicos de Lamport

Los relojes lógicos de Lamport permiten establecer un orden lógico de eventos en sistemas distribuidos sin depender del tiempo físico. Cada nodo mantiene un contador que se incrementa en cada evento interno o envío de mensaje. Al recibir un mensaje con timestamp T_r , el nodo actualiza su reloj local T_l según la regla:

$$T_l = \max(T_l, T_r) + 1$$

Este mecanismo establece relaciones de precedencia entre eventos concurrentes y constituye uno de los modelos fundamentales para sincronización distribuida [1], [2].

2.6. Medición de Latencia

La latencia es el tiempo que tarda un mensaje en viajar desde un emisor hasta un receptor. En sistemas distribuidos es un indicador clave de rendimiento: latencia baja implica transmisión rápida, latencia alta afecta negativamente el sistema [3]. Factores que la influyen incluyen tamaño de mensajes, velocidad de procesamiento, protocolo utilizado y mecanismos de serialización. Su análisis permite comparar tecnologías mediante métricas estadísticas: media, mediana, desviación estándar y percentil 95.

Factores como el protocolo utilizado, mecanismos de serialización, tamaño de mensaje y congestión de red influyen directamente en la latencia [4], [7]. Investigaciones recientes evidencian que gRPC puede reducir costos de serialización gracias a Protocol Buffers, mientras que TCP continúa destacando en escenarios de baja sobrecarga y comunicación directa [4], [5].

3. DESARROLLO DEL PROYECTO

3.1. Arquitectura General

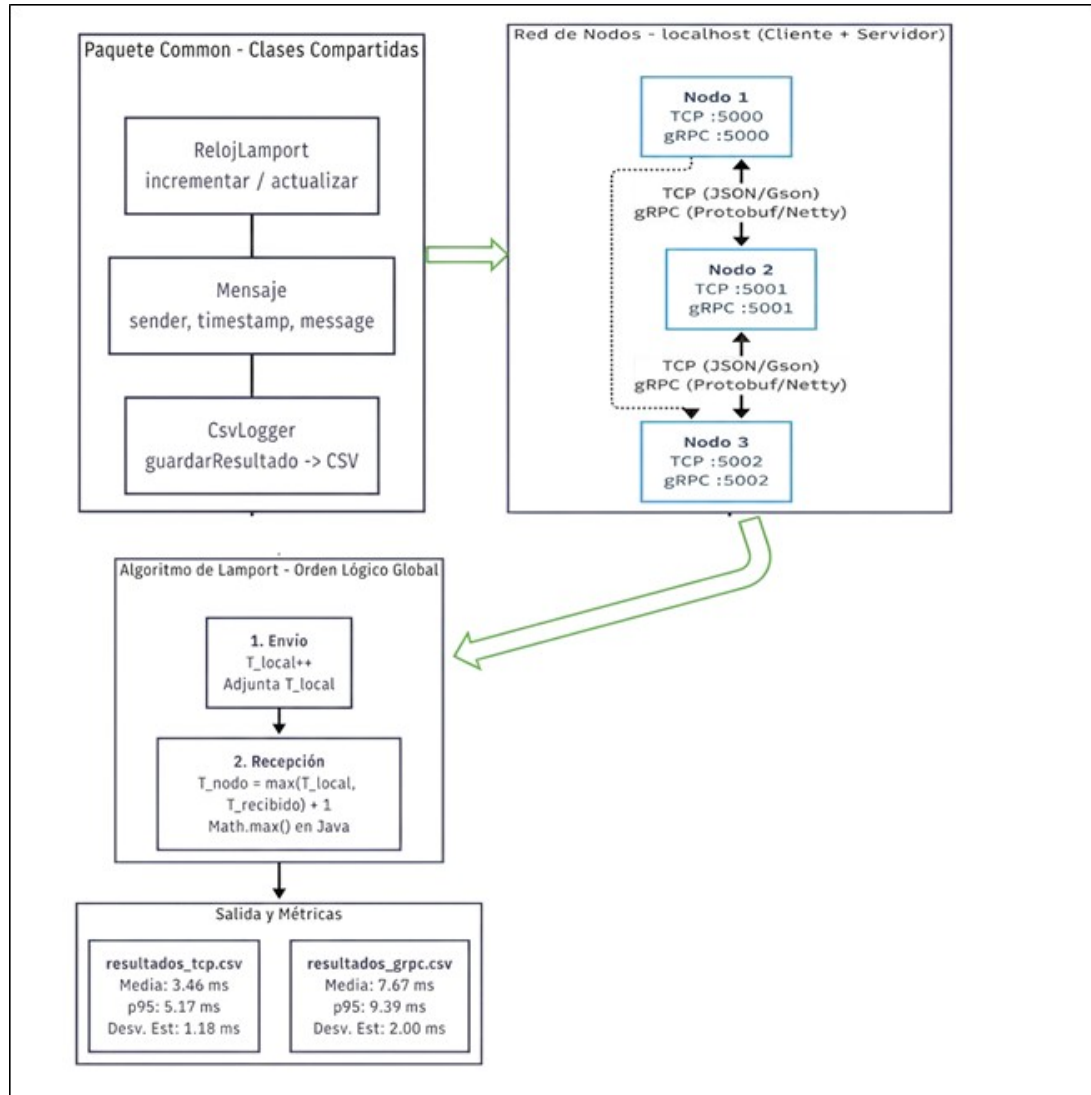


Figura 1: Arquitectura del proyecto

Tal como se observa en el diagrama de la Ilustración 1, el proceso inicia en el paquete común, pasa por la comunicación de la red de nodos en sus respectivos puertos, y finaliza con la ejecución del Algoritmo de Lamport para registrar los tiempos en archivos CSV.

El sistema está compuesto por tres nodos independientes que se comunican mediante sockets TCP y gRPC. Cada nodo puede actuar como emisor y receptor, permitiendo analizar el comportamiento de ambas tecnologías en un entorno distribuido controlado. En ambos modelos se implementaron relojes lógicos de Lamport: cada nodo incrementa su contador al enviar y lo actualiza al recibir mensajes. Los resultados se almacenaron en

archivos CSV para análisis estadístico posterior.

3.2. Implementación TCP

La implementación TCP emplea arquitectura cliente-servidor con tres nodos. El servidor permanece en escucha continua aceptando conexiones entrantes. Los mensajes son estructurados en formato JSON con tres campos: identificador del emisor, timestamp lógico de Lamport y contenido del mensaje. Cada cliente mantiene su propio reloj lógico, que incrementa antes de cada envío y actualiza al recibir, aplicando la regla de Lamport. Se realizaron múltiples intercambios para verificar sincronización y se midió latencia con 100 envíos consecutivos.

3.3. Implementación gRPC

La implementación gRPC define servicios y mensajes en un archivo .proto, a partir del cual se genera automáticamente el código de cliente y servidor. Los mensajes contienen los mismos tres campos: emisor, timestamp lógico y contenido. La serialización se realiza con Protocol Buffers y el transporte usa HTTP/2. El algoritmo de Lamport se aplica de forma idéntica al caso TCP. Se realizaron las mismas pruebas de 100 envíos para permitir comparación directa de rendimiento.

3.4. Configuración de Nodos

Los tres nodos se ejecutaron en entorno local. Los puertos asignados fueron:

- Nodo 1 = Puerto 5000
- Nodo 2 = Puerto 5001
- Nodo 3 = Puerto 5002

4. RESULTADOS EXPERIMENTALES

Los resultados obtenidos coinciden con investigaciones recientes sobre optimización de sockets TCP, donde se demuestra que TCP puede mantener menores tiempos de respuesta y alta estabilidad cuando existe una adecuada gestión de colas y buffers de transmisión [7].

4.1. Intercambios de Mensajes y Latencia Promedio

Se realizaron 20 intercambios de mensajes en ambas tecnologías, verificando el correcto envío, recepción y actualización de timestamps lógicos. Posteriormente se ejecutaron 100 envíos consecutivos para medir latencia promedio.

Cuadro 1: Intercambios de mensajes

Tecnología	Intercambios	Estado
TCP	20	Correcto
gRPC	20	Correcto

Los resultados reflejados en la Tabla 1 confirman que ambas tecnologías completaron con éxito el intercambio de los 20 mensajes programados, registrando un estado correcto tanto para TCP como para gRPC.

Cuadro 2: Latencia promedio por tecnología

Tecnología	Envíos	Latencia prom.
TCP	100	3.01 ms
gRPC	100	7.89 ms

De acuerdo con los datos recopilados en la Tabla 2, el rendimiento de TCP duplica en velocidad al de gRPC bajo la misma carga de 100 envíos, manteniendo un promedio de 3.01 ms frente a los 7.89 ms de este último.

Cuadro 3: Comparación general TCP vs. gRPC

Característica	TCP	gRPC
Tipo de comunicación	Cliente-Servidor	RPC
Protocolo	TCP	HTTP/2
Serialización	JSON	Protocol Buffers
Latencia promedio	Baja (3.01 ms)	Moderada (7.89 ms)
Complejidad	Media	Alta
Escalabilidad	Media	Alta
Rendimiento destacado	Menor latencia	Mayor estructura

Como se visualiza en la Tabla 3, el protocolo TCP destaca por ofrecer una menor latencia promedio (3.01 ms) y una complejidad media en comparación con gRPC, el cual, a pesar de registrar una latencia moderada (7.89 ms) y mayor complejidad, ofrece ventajas clave en escalabilidad y una estructura superior gracias al uso de HTTP/2 y Protocol Buffers.

4.2. Análisis Estadístico

Se aplicó análisis estadístico a los datos de latencia registrados en archivos CSV. Las métricas calculadas permiten comparar estabilidad, variabilidad y rendimiento de ambas tecnologías.

A pesar de ello, investigaciones recientes indican que gRPC logra mejores resultados en sistemas distribuidos de gran escala y arquitecturas basadas en microservicios debido a sus capacidades de multiplexación y serialización binaria eficiente [4], [5].

Cuadro 4: Estadísticas de latencia (ms)

Métrica	TCP	gRPC
Media	3.46 ms	7.67 ms
Mediana	3.01 ms	7.89 ms
Desviación estándar	1.18 ms	2.00 ms
Percentil 95	5.17 ms	9.39 ms

Los resultados de la tabla 4 muestran que TCP presenta menor latencia y mayor estabilidad (desviación estándar de 1.18 ms frente a 2.00 ms de gRPC). La sobrecarga adicional de gRPC —Protocol Buffers, llamadas remotas y HTTP/2— incrementa ligeramente el tiempo de respuesta, pero aporta ventajas en modularidad, escalabilidad y organización del sistema distribuido.

4.3. Análisis del Boxplot Comparativo

El boxplot confirmó visualmente los resultados estadísticos: TCP presenta menor dispersión y menor latencia promedio que gRPC. Los datos de muestra empleados en la Tabla 5 fueron:

Cuadro 5: Datos para BoxPlot

TCP (ms)	gRPC (ms)
2.5487	5.5547
5.5756	9.5515
3.5557	7.8924
2.6060	—
3.0086	—

La mayor variabilidad de gRPC no la descalifica: continúa siendo una alternativa eficiente y recomendable para arquitecturas de microservicios y sistemas distribuidos de gran escala.

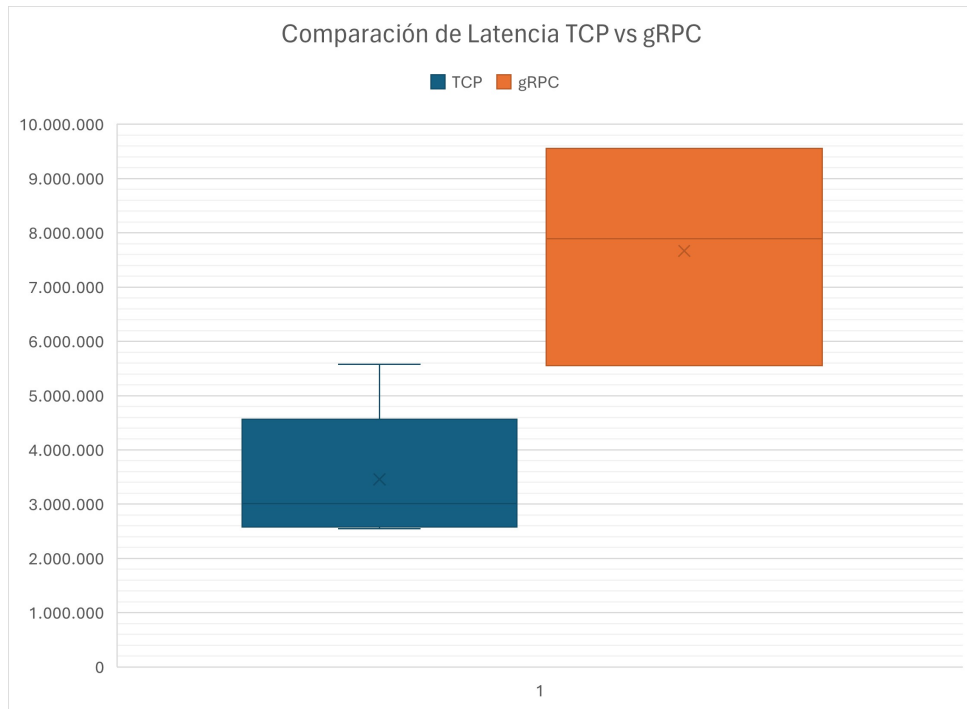


Figura 2: Grafico de BoxPlot

La Ilustración 2 presenta un gráfico de cajas (boxplot) que compara la latencia entre dos tecnologías de comunicación: TCP (en azul) y gRPC (en naranja). El eje vertical representa el tiempo de latencia en valores absolutos, mostrando que el protocolo TCP mantiene un rango de latencia notablemente inferior, con una media y una dispersión que oscilan entre los 2.500.000 y 5.500.000 unidades. Por el contrario, gRPC exhibe una latencia significativamente mayor y con una variabilidad más amplia, situándose en un rango aproximado de 5.500.000 a 9.500.000 unidades.

5. CONCLUSIONES

5.1. Freddy Vladimir Farinango Guandinango

El proyecto permitió comprender de forma práctica la comunicación en sistemas distribuidos. TCP demostró menor latencia gracias a su comunicación directa, mientras que gRPC ofreció arquitectura más organizada y moderna. Cada tecnología se adapta mejor según las necesidades del sistema: TCP para baja latencia y gRPC para entornos empresariales escalables. El trabajo fortaleció habilidades en redes, sincronización de procesos y programación distribuida.

5.2. Jeremy Ruperto Gaibor Rodríguez

El proyecto permitió aplicar conceptos de sistemas distribuidos y comunicación entre procesos, comprendiendo el papel de los relojes lógicos de Lamport en la sincronización de eventos. Los desafíos técnicos —conexiones entre nodos, configuración de puertos, manejo de mensajes— fortalecieron los conocimientos de programación en red. Los resultados confirman que la elección entre TCP y gRPC depende de los requerimientos de rendimiento, complejidad y escalabilidad del sistema.

5.3. Iván Andrés Villamarín Cuenca

El proyecto mostró las ventajas complementarias de ambas tecnologías: TCP logra mejores tiempos de respuesta por su menor sobrecarga, mientras que gRPC ofrece mayor eficiencia en serialización y facilidad de integración en arquitecturas modernas. La implementación práctica de relojes de Lamport facilitó la comprensión del orden lógico de eventos. El trabajo desarrolló habilidades en programación distribuida, análisis experimental y trabajo colaborativo.

6. BIBLIOGRAFIA

- [1] Z. Tong, S. Pakin, M. Lang, and X. Yuan, “Fast Classification of MPI Applications Using Lamport’s Logical Clocks,” *Proceedings - 2016 IEEE 30th International Parallel and Distributed Processing Symposium, IPDPS 2016*, pp. 618–627, Jul. 2016, doi: 10.1109/IPDPS.2016.40.
- [2] R. L. Probert, H. Yu, and K. Saleh, “Relative-clock-based specification and test result analysis of distributed systems,” *11th Annual International Phoenix Conference on Computers and Communication, IPCCC 1992 - Proceedings*, pp. 687–694, 1992, doi: 10.1109/PCCC.1992.200507.
- [3] C. Yu, H. Zhou, X. Lu, J. Lai, and G. P. Liu, “Distributed Optimal Synchronization Rate Control for AC Microgrids under Event-Triggered Mechanism,” *IEEE Transactions on Power Systems*, vol. 36, no. 3, pp. 1780–1793, May 2021, doi: 10.1109/TPWRS.2020.3035688.
- [4] G. Swapna, S. V. Punith Kumar, and E. Susmitha, “Relix: A Comparative Latency Framework for REST and GRPC Using Redis Acceleration,” *International Research Journal on Advanced Engineering Hub (IRJAEH)*, vol. 4, no. 05, pp. 3395–3401, May 2026, doi: 10.47392/IRJAEH.2026.0438.
- [5] M. Niswar, R. A. Safruddin, A. Bustamin, and I. Aswad, “Performance evaluation of microservices communication with REST, GraphQL, and gRPC,” *International Journal of Electronics and Telecommunications*, vol. 70, no. 2, pp. 429–436, Jun. 2024, doi:

10.24425/IJET.2024.149562.

[6] C. Carthen, A. Zaremehrijardi, Z. Estreito, A. Tavakkoli, F. C. Harris, and S. M. Dascalu, “SpeciServe. a gRPC Infrastructure Concept; SpeciServe. a gRPC Infrastructure Concept,” 2024, doi: 10.1109/SERA61261.2024.10685565.

[7] H. H. Park and Z. Meng, “Poster: Controlling TCP Socket Queueing Delay,” SIGCOMM Posters and Demos 2024 - Proceedings of the 2024 SIGCOMM Poster and Demo Sessions, Part of SIGCOMM 2024, pp. 63–65, Aug. 2024, doi: 10.1145/3672202.3673745.

[8] A. Goel and B. Thangaraju, “Authenticating Distributed Systems Using SPIRE over Kubernetes Cluster,” 2022 IEEE International Conference on Electronics, Computing and Communication Technologies, CONECCT 2022, 2022, doi: 10.1109/CONECCT55679.2022.9865835.

7. ANEXOS

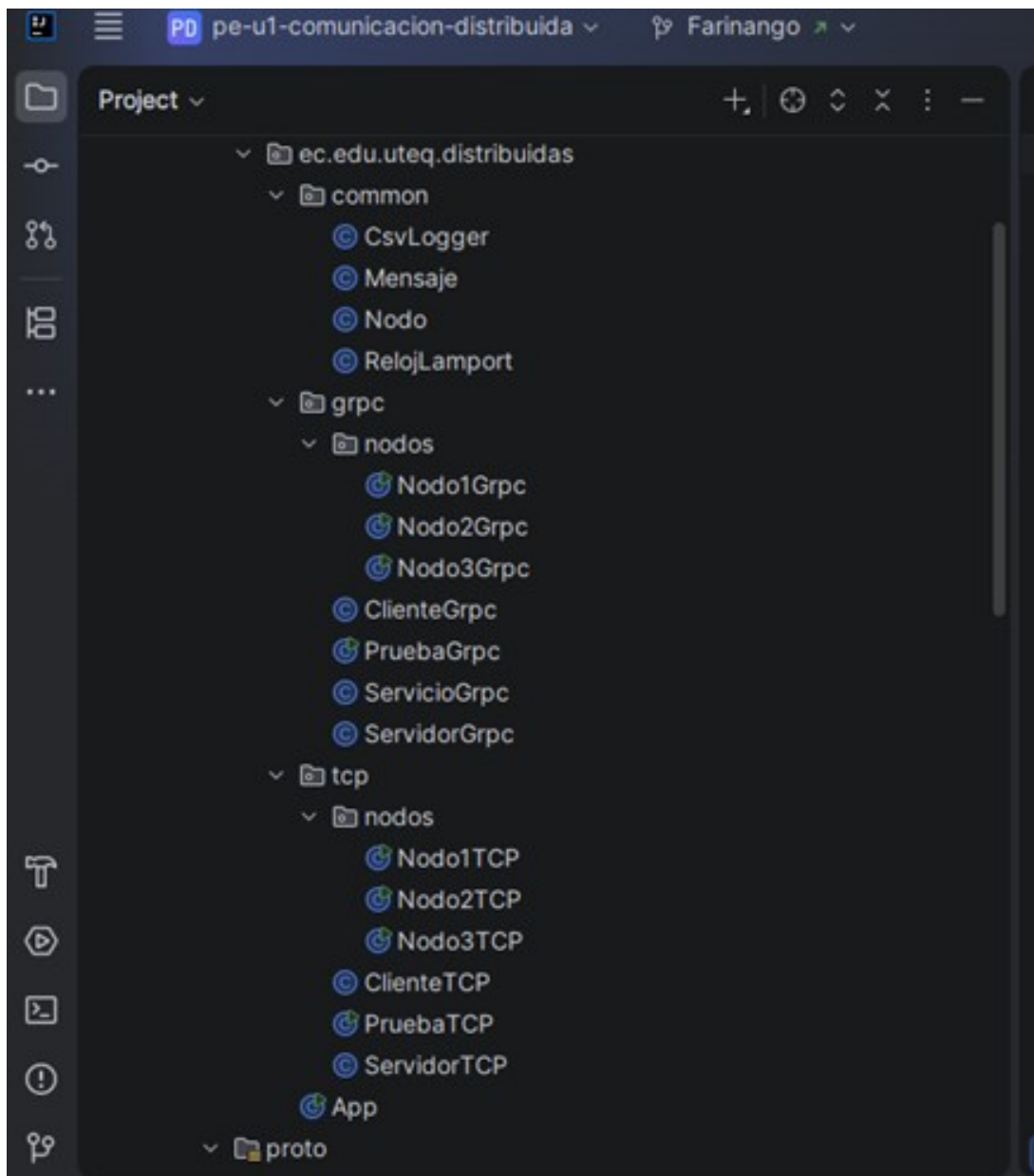


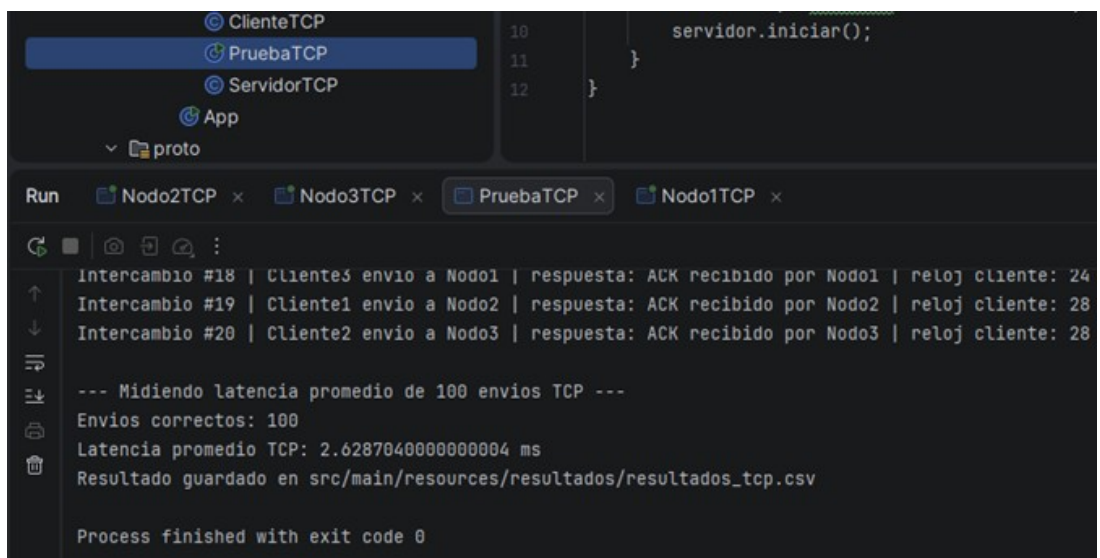
Figura 3: Estructura general del proyecto



```
Run  Nodo1Grpc x  Nodo2Grpc x  Nodo3Grpc x  PruebaGrpc x
Intercambio #19 | Cliente1 envio a Nodo2 | respuesta: ACK gRPC recibido por Nodo2 | reloj cliente: 28
Intercambio #20 | Cliente2 envio a Nodo3 | respuesta: ACK gRPC recibido por Nodo3 | reloj cliente: 28
--- Midiendo latencia promedio de 100 envios gRPC ---
Envios correctos: 100
Latencia promedio gRPC: 7.936744999999999 ms
Resultado guardado en src/main/resources/resultados/resultados_grpc.csv

Process finished with exit code 0
```

Figura 4: Resultado de ejecución prueba GRPC



```
Run  Nodo2TCP x  Nodo3TCP x  PruebaTCP x  Nodo1TCP x
Intercambio #18 | Cliente3 envio a Nodo1 | respuesta: ACK recibido por Nodo1 | reloj cliente: 24
Intercambio #19 | Cliente1 envio a Nodo2 | respuesta: ACK recibido por Nodo2 | reloj cliente: 28
Intercambio #20 | Cliente2 envio a Nodo3 | respuesta: ACK recibido por Nodo3 | reloj cliente: 28
--- Midiendo latencia promedio de 100 envios TCP ---
Envios correctos: 100
Latencia promedio TCP: 2.6287040000000004 ms
Resultado guardado en src/main/resources/resultados/resultados_tcp.csv

Process finished with exit code 0
```

Figura 5: Resultado de ejecución Prueba TCP